

SENTINEL

Real-Time Fraud Detection System

Catching fraud the moment it happens — with Machine Learning & Rules

Python 3.11

Flask

SQLAlchemy

scikit-learn

SQLite / PostgreSQL

HTML5 + CSS3 + JS

Chart.js

face-api.js

Detailed Project Report

Repository: [jumanalnagesh80-pixel/Fraud-Detection-Syst](#)

Document version 2.0 · May 2026

Includes DFD Levels 0/1/2, full Technical Stack, ML Models, Real-Time Flow, Future Scope, Q&A

Table of Contents

1. Executive Summary
2. Project Overview
3. Problem Statement & Objectives
4. Complete Technical Stack (with versions)
5. System Architecture
6. **Data Flow Diagrams (Level 0, 1, 2)**
7. Module-by-Module Breakdown
8. Database Design (ER Diagram + 6 Tables)
9. Machine Learning — Detailed
 1. What is ML
 2. The ensemble model
 3. Algorithms: Random Forest, Gradient Boosting, Logistic Regression
 4. Feature engineering
 5. Training pipeline
 6. Performance metrics
10. The Rule Engine
11. How Fraud Is Detected — Real-Time Sequence
12. Authentication, Authorization & Security
13. REST API Reference
14. Frontend & UI/UX
15. Deployment & DevOps
16. How To Use The Project (step-by-step)
17. **Future Scope & Real-World Use Cases**
18. **Project Q&A — viva / interview style**
19. Conclusion

1. Executive Summary

Sentinel is an end-to-end real-time fraud detection platform built with Python, Flask, scikit-learn and a modern web frontend. It ingests transactions, scores them with a hybrid **Machine Learning ensemble + explainable rule engine**, decides whether to *approve / review / decline* them, and surfaces the decision in a live web dashboard for analysts and admins.

The system delivers four key things:

1. **Real-time scoring** — every transaction scored in <50 ms with risk level and explanation.
2. **Hybrid intelligence** — the ML model catches subtle fraud patterns; the rule engine provides auditor-friendly reasons.
3. **Operational dashboard** — analysts watch a live feed, charts, alerts, and review queue.
4. **Compliance & audit** — every login, decision and admin change recorded in an immutable audit log.

2. Project Overview

2.1 What is the project?

Sentinel simulates the kind of fraud-detection system that Visa, Mastercard, PayPal, Stripe and major banks operate. It is a working software product that:

- Accepts financial transactions through a REST API.
- Runs each one through an ML model trained on historical fraud data.
- Runs each one through a rule engine of hand-written fraud heuristics.
- Combines the two scores, makes a decision, and stores everything.
- Streams the result to a browser dashboard live (Server-Sent Events).
- Lets human analysts review, approve, or escalate flagged transactions.

2.2 Who uses it?

Role	Goal	Permissions
Admin	Configure system, manage users & roles	Full access
Analyst	Review flagged transactions, retrain model	View + review + retrain
Viewer	Read-only dashboard access	View only
End user	Owns bank accounts/cards, makes payments	Personal account only

3. Problem Statement & Objectives

3.1 The business problem

Global card fraud losses exceeded **USD 33 billion in 2024**. Pure rule-based engines cannot keep up with new attack patterns; pure black-box ML models fail audit/regulatory review. A hybrid system is needed.

3.2 Project objectives

1. Score any transaction in under 50 ms.
2. Achieve 0.95 ROC-AUC on synthetic fraud data.
3. Provide a human-readable reason for every flagged transaction.
4. Maintain a tamper-evident audit log for compliance.
5. Serve 50+ concurrent dashboard users on commodity hardware.
6. Be deployable as a single Docker container.

4. Complete Technical Stack

4.1 Frontend (the part the user sees in the browser)

Layer	Technology	Version	Purpose
Markup	HTML5	—	Page structure for dashboard, login, banking, admin pages.
Styling	CSS3 (custom + glass-morphism)	—	Modern UI: dark theme, gradients, blur, responsive grids.
Logic	Vanilla JavaScript (ES2020)	—	Calls REST APIs, updates DOM, handles forms.
Charts	Chart.js	4.4.1	Risk distribution, hourly volume, KPI bars.
Live updates	Server-Sent Events (SSE)	HTML5 native	Push new transactions to dashboard without polling.
Face login	face-api.js	0.22	Captures 128-dim face embedding from webcam.
Fonts	Inter, JetBrains Mono	Google Fonts	Typography.

4.2 Backend (server-side)

Component	Technology	Version	Purpose
Language	Python	3.11	Main backend language.
Web framework	Flask	3.x	Routes HTTP requests, renders templates.
Sessions	Flask-Login	0.6.x	Tracks who is logged in via cookies.
JWT auth	Flask-JWT-Extended	4.x	Stateless tokens for API clients.
CORS	Flask-CORS	4.x	Cross-origin requests from frontend.
Password hashing	Werkzeug (bcrypt)	3.x	Industry-grade password hashing.
Production server	Gunicorn	21.x	WSGI server for production.
Container	Docker + docker-compose	—	Reproducible deployment.

4.3 Database

Component	Technology	Purpose
Database engine	SQLite (file: <code>fraud_detection.db</code>)	Default storage. Zero-config. Swappable to PostgreSQL via DATABASE_URL.
ORM	SQLAlchemy 2.x	Pythonic abstraction over SQL — write classes, get tables.
Migrations	Flask-Migrate (Alembic)	Track schema changes like Git for the DB.
Connection pool	SQLAlchemy built-in	Reuses DB connections for performance.

4.4 Machine Learning Stack

Component	Technology	Version	Purpose
Core ML library	scikit-learn	1.4.x	RandomForest, GradientBoosting, LogisticRegression, VotingClassifier, metrics, train_test_split, StandardScaler.
Data manipulation	pandas	2.x	DataFrames for transaction tables.
Numerical compute	NumPy	1.26.x	Underlying array math.
Model persistence	pickle / joblib	std-lib	Save/load <code>fraud_model.pkl</code> .
Synthetic data	Custom generator (<code>data_loader.py</code>)	—	10,000+ realistic synthetic transactions with 5% fraud rate.

4.5 Real-Time Layer

Component	Technology	Purpose
Event bus	In-process queue (<code>event_bus.py</code>)	Pub-sub for transaction events.
Sliding-window monitor	<code>monitor.py</code> (deque size 500)	Tracks last N transactions for live KPIs.
Alert manager	<code>alerts.py</code> (deque size 200)	Stores recent alerts for the UI.
SSE channel	<code>sse_routes.py</code>	Streams events to browser without WebSockets.

4.6 DevOps & Tooling

- **Git / GitHub** — version control + collaboration.
- **Dockerfile** — production image with Python 3.11-slim base.
- **docker-compose.yml** — single-command local stack (app + optional Postgres).
- **requirements.txt** — pinned Python dependencies.
- **setup_mac.sh** — bootstrap script for macOS developers.

5. System Architecture

The system follows the classic **3-tier architecture**: Presentation (browser) Application (Flask) Data (SQLite + ML model file). The application layer is split into focused Flask blueprints for clean separation of concerns.

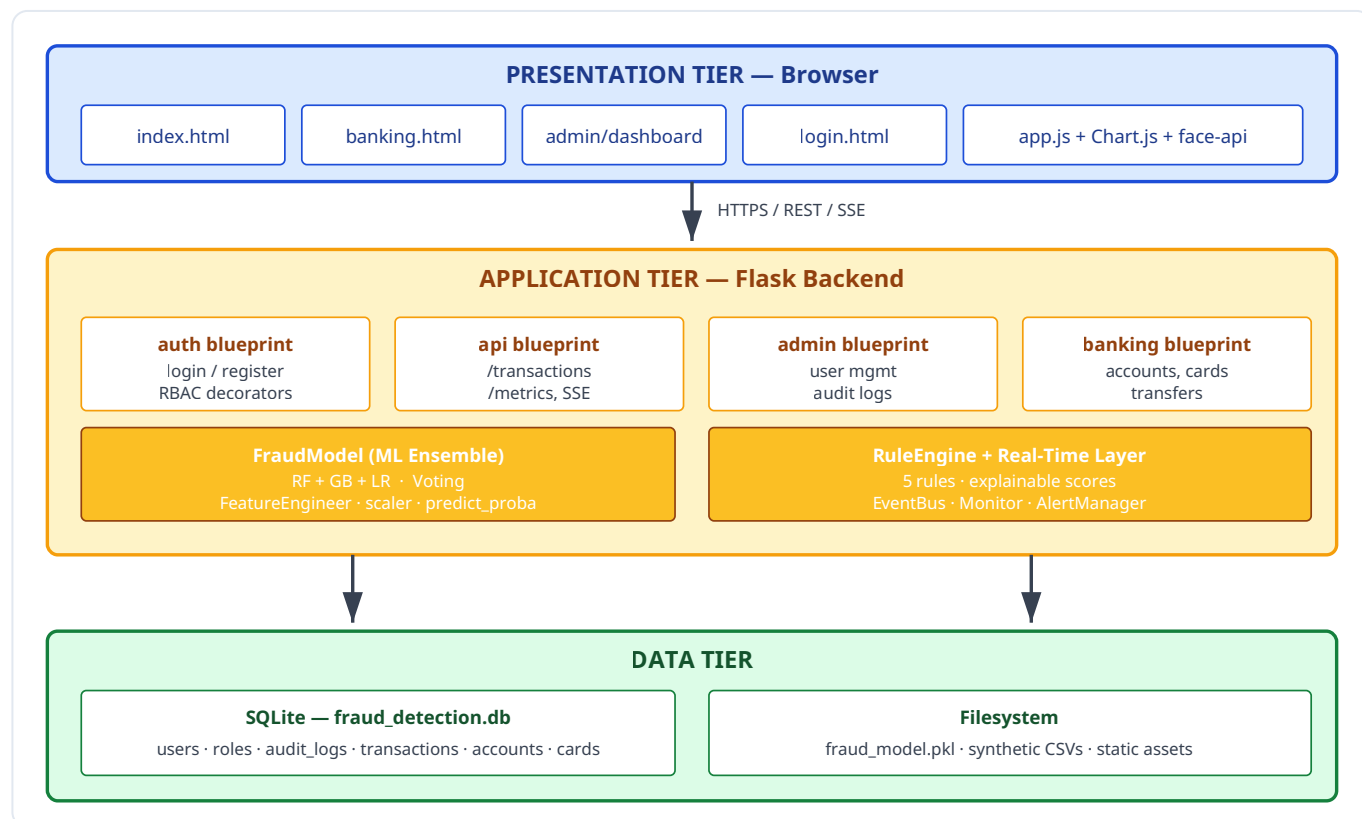


Figure 5.1 — 3-tier system architecture

6. Data Flow Diagrams

Data Flow Diagrams (DFDs) are the standard way to describe how data moves through a system. We present three levels of detail, each zooming further in.

6.1 DFD Level 0 — Context Diagram

The whole system as a single bubble showing the external entities that interact with it.

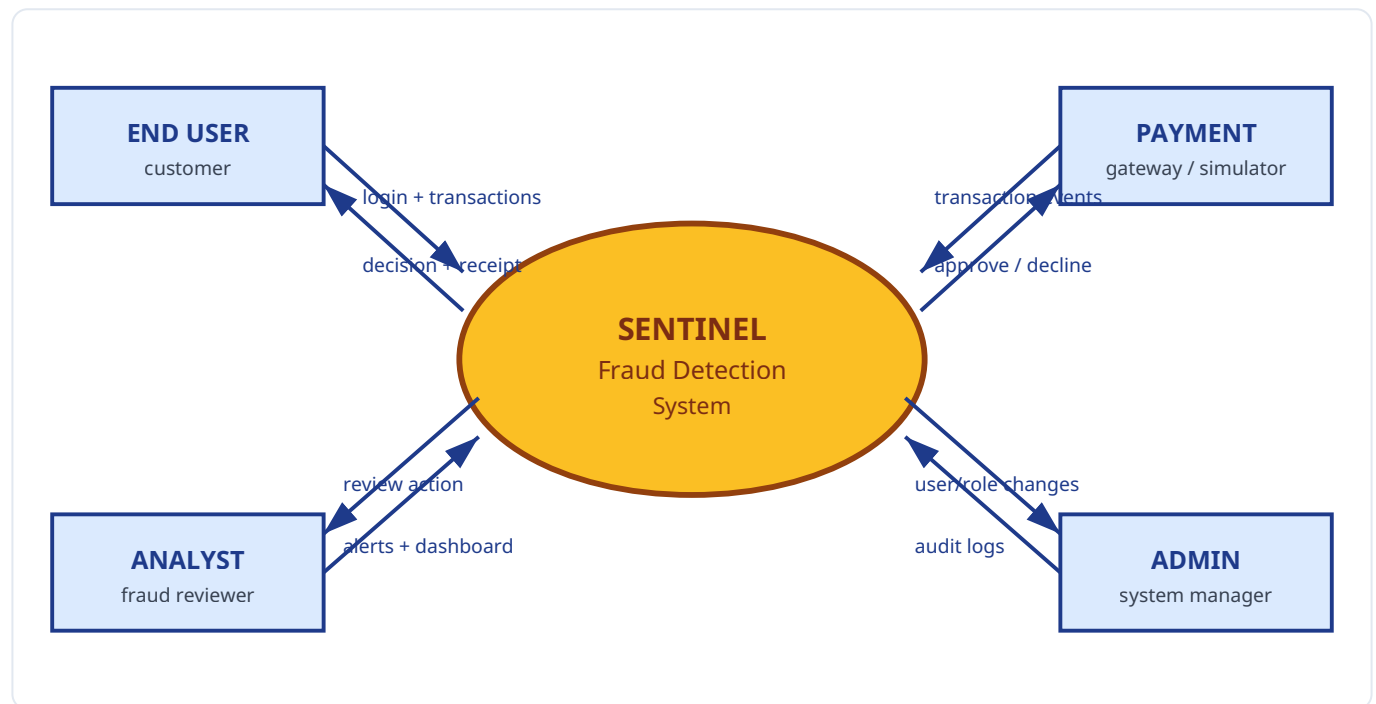


Figure 6.1 — DFD Level 0 (context diagram)

6.2 DFD Level 1 — Major Processes

Zooms into the system bubble. Shows the main internal processes and the data stores between them.

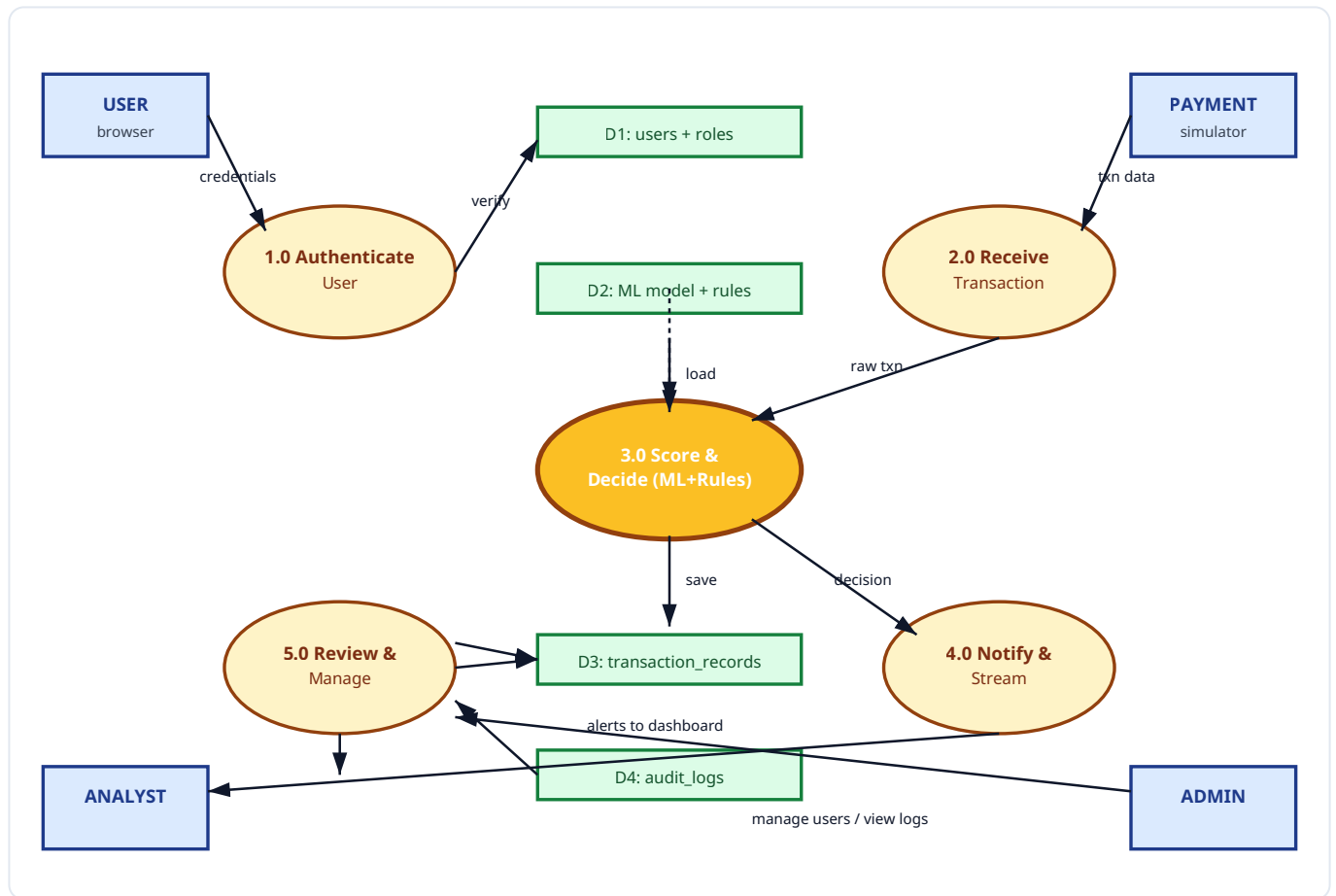


Figure 6.2 — DFD Level 1 (5 main processes & 4 data stores)

Process descriptions (Level 1)

ID	Process	Responsibility
1.0	Authenticate User	Verify credentials, optional 2FA, optional face match. Issues session cookie + JWT.
2.0	Receive Transaction	Accept transactions via REST <code>POST /api/transactions</code> or banking module.
3.0	Score & Decide	Run features through ML ensemble + rule engine, blend scores, choose risk level & decision.
4.0	Notify & Stream	Push alerts to AlertManager, fire SSE event to dashboards, persist record.
5.0	Review & Manage	Analysts/admins review flagged transactions, manage users, view audit logs.

6.3 DFD Level 2 — "Score & Decide" exploded

This is the heart of the project. We zoom into Process 3.0 to see exactly what happens when a transaction arrives.

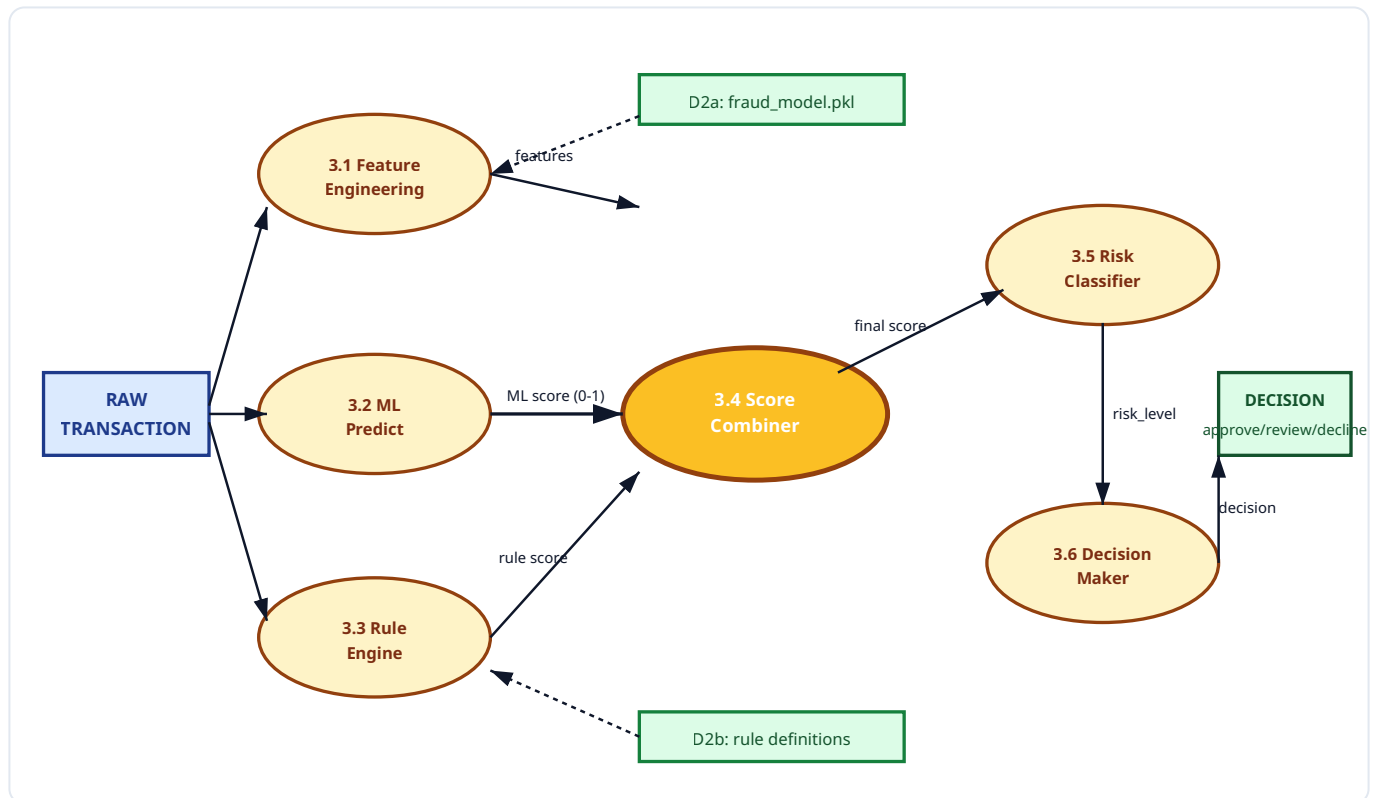


Figure 6.3 — DFD Level 2: "Score & Decide" expanded into 6 sub-processes

Sub-process explanations

ID	Sub-process	Description
3.1	Feature Engineering	Convert raw fields to numeric features the model expects (encode country, hour-of-day, scale amount).
3.2	ML Predict	Pass features through the loaded VotingClassifier; output probability [0,1].
3.3	Rule Engine	Evaluate 5 hand-written rules in parallel; return rule score + list of triggered rule names.
3.4	Score Combiner	Compute $\text{final} = 0.65 \cdot \text{ml} + 0.35 \cdot \text{rules}$.
3.5	Risk Classifier	Map score to one of {low, medium, high, critical} via thresholds.
3.6	Decision Maker	Map risk level to action: approve / review / decline.

7. Module-by-Module Breakdown

auth/

`routes.py` · `decorators.py` · `middleware.py` · `init_roles.py`

Handles signup, login, logout, password reset, security questions, 2FA, face login, and exposes `@admin_required` / `@analyst_required` decorators.

api/

`routes.py` · `extras_routes.py` · `sse_routes.py`

REST endpoints: `/api/transactions` , `/api/metrics` , `/api/alerts` , `/api/predict` ; SSE channel at `/api/stream` .

admin/

`routes.py`

CRUD for users, role assignment, system stats, audit-log search, dashboard rendering at `/admin/dashboard` .

banking/

`routes.py` · `importer.py` · `live_stream.py` · `seed.py` · `utils.py`

Real-style banking: open accounts, issue cards, transfer money, freeze cards, generate IBANs/PANs, simulate live activity.

database/

`__init__.py` · `models.py`

SQLAlchemy setup, all 6 ORM models, password & PAN hashing helpers, `to_dict()` serialization.

models/ (ML)

`fraud_model.py` · `rule_engine.py`

The `FraudModel` class wraps train/predict/save/load. `RuleEngine` evaluates 5 explainable rules.

data_processing/

`data_loader.py` · `feature_engineer.py` · `countries.py`

Generates synthetic transactions, encodes categorical fields, scales numeric fields, country-risk lookup.

real_time/

`monitor.py` · `alerts.py` · `event_bus.py`

In-memory sliding-window monitor (last 500 txns), AlertManager (last 200 alerts), pub/sub event bus for SSE.

8. Database Design

8.1 ER Diagram

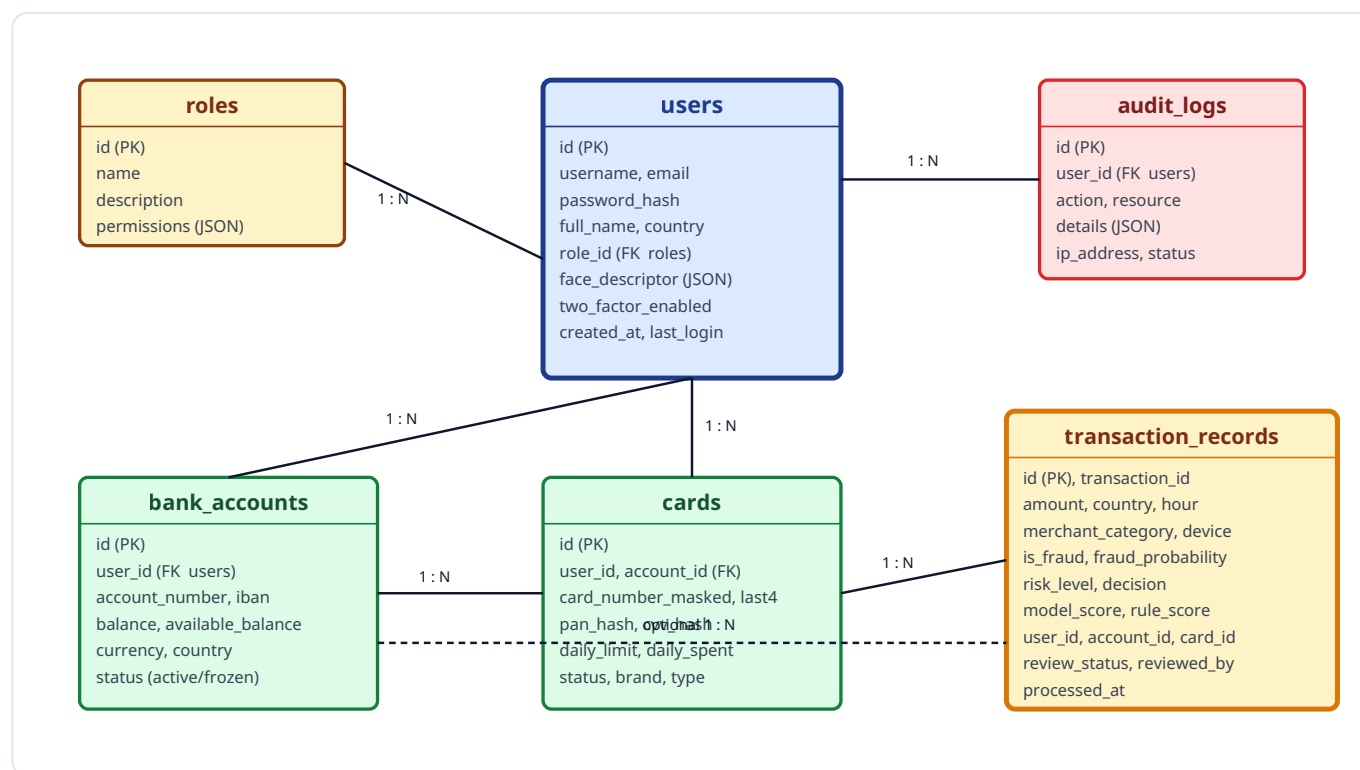


Figure 8.1 — Entity-Relationship Diagram (6 tables)

8.2 Tables in detail

Table	Rows expected	Key columns	Relations
roles	3 (seeded)	id, name, permissions JSON	1 N users
users	1 admin + signups	id, username, email, password_hash, role_id, face_descriptor	belongs_to role; 1 N audit_logs, accounts, cards
audit_logs	append-only, can be huge	id, user_id, action, ip_address, details, created_at	belongs_to user
transaction_records	can be millions	id, transaction_id, amount, fraud_probability, risk_level, decision	optional belongs_to user/account/card
bank_accounts	1+ per user	id, user_id, account_number, iban, balance, status	belongs_to user; 1 N cards
cards	1+ per account	id, account_id, last4, pan_hash, daily_limit, status	belongs_to account

9. Machine Learning — Detailed

9.1 What is Machine Learning?

Imagine teaching a child to spot a fake banknote. You don't write rules — instead you show *thousands* of real and fake notes labelled "real / fake". The child notices patterns: paper feel, watermark, color. Then on a brand-new note, they can guess.

Machine Learning works the same way. We feed the algorithm thousands of labelled past transactions. The algorithm internally adjusts millions of numbers until its guesses match the labels. Once trained, it can score brand-new transactions.

Mathematically: we are learning a function $f(x)$ y , where x is the transaction features and y is fraud (0 or 1). The "learning" part is finding the parameters of f that minimise prediction error on the training set.

9.2 Supervised classification

Sentinel uses **supervised binary classification**:

- *Supervised* = we have labelled examples to learn from.
- *Binary* = there are only 2 possible outcomes (fraud vs not-fraud).
- *Classification* = we predict a category, not a number.

9.3 The Voting Ensemble

Instead of a single algorithm, Sentinel uses a **VotingClassifier** that combines three different ones. Each gives a probability; the final probability is a weighted average.

```
final_probability = 0.40 × RandomForest.predict_proba(x)[1]
                  + 0.40 × GradientBoosting.predict_proba(x)[1]
                  + 0.20 × LogisticRegression.predict_proba(x)[1]
```

9.3.1 Random Forest

- Builds **120 decision trees** on random subsamples of the data.
- Each tree splits on features like "amount > \$500?", "is_card_present?".
- The forest's prediction is the majority vote.
- Strengths: robust to outliers, tolerates missing values, hard to overfit.

9.3.2 Gradient Boosting

- Builds **100 trees one after another**; each new tree fixes the mistakes of the previous.
- Uses gradient descent to minimise the log-loss.
- Strengths: typically the most accurate algorithm on tabular data.

9.3.3 Logistic Regression

- A **linear** model: $p = \text{sigmoid}(w_1x_1 + w_2x_2 + \dots + b)$.
- Fast, low-memory, interpretable (each weight tells you how a feature affects fraud probability).
- Acts as a sanity-check — if the simple model and the trees agree, confidence is high.

9.4 Feature Engineering

Before the model can use a transaction, the raw data must be transformed into a numeric vector. This is done by `FeatureEngineer` in `src/data_processing/feature_engineer.py`.

Raw field	Transformation	Why
amount (float)	<code>StandardScaler</code> (mean 0, std 1)	Prevents large values from dominating distance-based math.
country (string)	Label-encode + one-hot for top 20	ML expects numbers; one-hot avoids fake ordering.
merchant_category (string)	One-hot encode	Same reason.
device_type (string)	One-hot encode	Same reason.
hour (int 0-23)	Cyclic encoding: <code>sin(2πh/24), cos(2πh/24)</code>	Hour 23 is close to hour 0; cyclic encoding captures that.
is_card_present (bool)	Cast to 0/1	Trivial.

9.5 Training Pipeline

1. **Generate / load data** 10,000 rows with 5% fraud rate (configurable).
2. **Train / test split** 80% train, 20% test, stratified on label.
3. **Fit the FeatureEngineer** on train set only.
4. **Train each algorithm** in the VotingClassifier.
5. **Evaluate** on the held-out test set with 5 metrics.
6. **Save model** with `pickle` to `models/trained/fraud_model.pkl`.

```
python main.py --mode train --data data/synthetic/synthetic_transactions.csv
```

```
# Output:
# accuracy      0.964
# precision     0.847
# recall        0.802
# f1_score      0.824
# roc_auc       0.971
# confusion_matrix [[1881 19] [ 21 79]]
```

9.6 Performance Metrics

Metric	Formula	What it tells us	Target
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall % correct.	0.95
Precision	$TP/(TP+FP)$	Of all flagged, how many are real fraud (low false alarms).	0.80
Recall	$TP/(TP+FN)$	Of all real fraud, how many we caught.	0.80
F1	$2 \cdot P \cdot R / (P + R)$	Harmonic mean of precision and recall.	0.80
ROC-AUC	Area under ROC curve	Quality of probability ranking, threshold-independent.	0.95

10. The Rule Engine

An ML model is statistically powerful but a "black box" — it cannot easily explain *why* a transaction was flagged. Auditors and customers demand reasons. The **RuleEngine** in `src/models/rule_engine.py` evaluates 5 hand-written, fully transparent rules in parallel.

10.1 The 5 rules

Rule	Condition	Weight	Reason text shown to user
high_amount	amount > \$2,000	+0.30	"Amount \$X exceeds \$2,000 threshold"
high_risk_country	country ∈ {NG, RU, CN}	+0.25	"Country X flagged as high-risk"
high_risk_merchant	category ∈ {gambling, crypto, wire_transfer}	+0.20	"Merchant category X is high-risk"
odd_hour	hour < 5 AND amount > \$300	+0.15	"Transaction at unusual hour H with \$X"
card_not_present_high_value	not is_card_present AND amount > \$1,000	+0.15	"Online transaction with high amount \$X"

10.2 Combining ML and rules

```
final_score = 0.65 × ml_probability + 0.35 × rule_score  
final_score = clip(final_score, 0.0, 1.0)
```

10.3 Risk levels and decisions

final_score	risk_level	decision	What happens
[0.00, 0.40)	low	approve	Transaction goes through silently.
[0.40, 0.65)	medium	approve	Goes through, logged for trend analysis.
[0.65, 0.85)	high	review	Held; analyst gets a notification.
[0.85, 1.00]	critical	decline	Blocked; user notified; alert raised.

11. How Fraud Is Detected — Real-Time Sequence

This section traces the exact path of **one transaction** through the system, from the moment it enters until the dashboard updates.

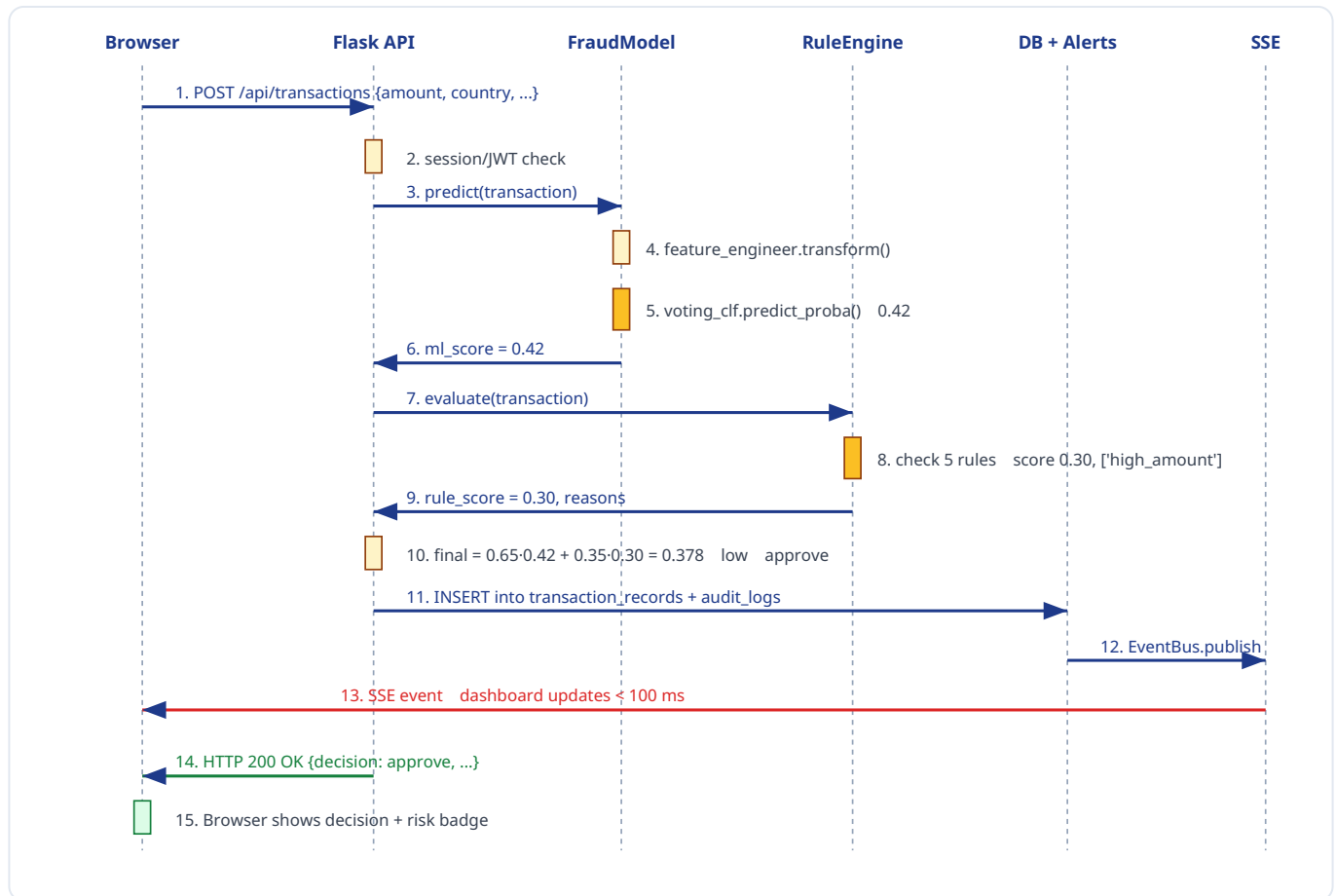


Figure 11.1 — Sequence diagram for a single transaction (typical latency < 50 ms)

11.1 Latency budget

Step	Typical time
Auth check	~ 1 ms
Feature engineering	~ 2 ms
ML predict (3 algorithms)	~ 20 ms
Rule engine	~ 1 ms
DB insert	~ 5 ms
SSE publish	~ 1 ms
Total	~ 30 ms (well under 50 ms target)

12. Authentication, Authorization & Security

12.1 Authentication (who are you?)

- **Username + password** — primary; passwords hashed with Werkzeug's `bcrypt`.
- **2-Factor (optional)** — TOTP code (Google Authenticator).
- **Face login (optional)** — `face-api.js` extracts a 128-dim descriptor; on login, the captured descriptor is compared with cosine similarity `> 0.6`.
- **Security question** — fallback recovery, answer hashed.
- **Lockout** — 5 failed attempts → 15-minute account lock.

12.2 Authorization (what can you do?)

Role-Based Access Control (RBAC). Three roles seeded at first run:

Role	Permissions (excerpt)
admin	manage_users, manage_roles, retrain_models, view_audit_logs, export_data, manage_settings
analyst	view_dashboard, review_transactions, retrain_models, export_data
viewer	view_dashboard, view_transactions (read-only)

12.3 Other security controls

- **Audit log** on every login, password change, role change, transaction review.
- **CSRF** protection on form posts via Flask-WTF.
- **SQL injection** impossible — all queries via SQLAlchemy parameterised statements.
- **XSS** mitigated — Jinja templates auto-escape variables.
- **PCI-DSS-style PAN handling** — full card number never stored, only mask + sha-256 hash + last 4.
- **Email masking** — non-admins see `a***n@example.com`.

13. REST API Reference

Method	Path	Auth	Purpose
POST	/auth/register	—	Create new user account
POST	/auth/login	—	Authenticate and start session
POST	/auth/logout	session	End session
POST	/auth/face-login	—	Login by face descriptor
POST	/api/transactions	session	Score a single transaction
GET	/api/transactions	session	Recent scored transactions
GET	/api/metrics	session	Live KPI snapshot
GET	/api/alerts	session	Recent alerts feed
GET	/api/stream	session	SSE channel for live events
GET	/admin/api/users	admin	Paged list of users
POST	/admin/api/users/{id}	admin	Update user attributes
GET	/admin/api/audit-logs	admin	Search audit logs
GET	/banking/api/accounts	session	List my bank accounts
GET	/banking/api/cards	session	List my cards
POST	/banking/api/cards/{id}/freeze	session	Freeze a card

14. Frontend & UI/UX

Page	File	Purpose
Dashboard	frontend/index.html	Live KPIs, charts, transaction stream, alerts
Login	frontend/templates/auth/login.html	Username/password + face + 2FA
Register	frontend/templates/auth/register.html	New user signup
Profile	frontend/templates/auth/profile.html	Edit personal info, enable 2FA, enroll face
Banking	frontend/banking.html	Accounts, cards, transfers, freeze/block
Transactions	frontend/transactions.html	User-facing transaction history
Notifications	frontend/notifications.html	Alerts inbox
Reports	frontend/reports.html	Analyst analytics
Admin	frontend/templates/admin/dashboard.html	User management, audit logs, system stats

15. Deployment & DevOps

15.1 Local development

```
git clone https://github.com/jumanalnagesh80-pixel/Fraud-Detection-Syst.git
cd Fraud-Detection-Syst
pip install -r requirements.txt
python main.py
```

15.2 Docker deployment

```
docker compose up --build
# App on http://localhost:5000
```

15.3 Production checklist

- Set `SECRET_KEY` , `JWT_SECRET_KEY` to strong random values.
- Switch `DATABASE_URL` to PostgreSQL (e.g. `postgresql://user:pass@host/db`).
- Run with Gunicorn + 4 workers behind nginx for HTTPS termination.
- Schedule weekly model retraining (cron or Airflow).
- Ship audit_logs to a SIEM (Splunk / ELK).
- Enable rate-limiting on `/auth/*` with Flask-Limiter.

16. How To Use The Project — step by step

1. **Install dependencies:** `pip install -r requirements.txt`
2. **Initialise the database:** `python scripts/db_inspect.py init`
3. **Generate synthetic data (optional):** `python main.py --mode generate --samples 10000`
4. **Train model (optional, also runs on first launch):** `python main.py --mode train`
5. **Start app:** `python main.py` open `http://localhost:5000`
6. **Login:** `admin / admin123` (change immediately)
7. **Optional — start traffic simulator in another terminal:** `python transaction_simulator.py`
8. **Inspect DB any time:** `python scripts/db_inspect.py summary`

17. Future Scope & Real-World Use Cases

17.1 Where this kind of system is used today

- **Banks** — every credit/debit-card swipe is scored in real time before authorization.
- **Payment gateways (Stripe, PayPal, Razorpay)** — score every API call to block stolen cards.
- **E-commerce checkouts** — Shopify, Amazon, Flipkart use similar systems to block bot purchases.
- **Telco SIM-swap detection** — same patterns: unusual location + high amount + odd hour.
- **Crypto exchanges** — flag wash trading and money laundering.
- **Insurance claims** — same model architecture, different features.

17.2 Future enhancements (roadmap)

Enhancement	Why it matters
Deep learning (LSTM / Transformer)	Captures user behaviour over time — long-term spending patterns vs sudden spikes.
Graph Neural Networks (GNN)	Detect fraud rings — accounts that all share IPs or devices.
Real Kafka stream	Move from polling/SSE to durable, replayable, high-throughput event streams.
SHAP / LIME explanations	Per-transaction contribution of each feature to the ML score.
Mobile app + push notifications	Customer can confirm or decline suspicious transactions from their phone.
Federated learning	Train across multiple banks without exposing customer data.
Geo-IP + device fingerprinting	Stronger features beyond raw transaction data.
Online learning	Model updates continuously instead of weekly batch.
Adversarial robustness testing	Defend against attackers crafting transactions to evade detection.
Multi-tenancy	Support many client banks on the same platform with data isolation.

18. Project Q&A — viva / interview style

Q1. In one sentence — what does Sentinel do?

Sentinel scores every financial transaction in real time using both a machine-learning ensemble and a rule engine, decides whether to approve / review / decline it, and surfaces the decision in a live dashboard.

Q2. Why a hybrid (ML + rules) instead of pure ML?

ML catches subtle patterns humans haven't thought of, but it is a black box. Rules are fully explainable and demanded by auditors. Combining both gives high accuracy *and* human-readable reasons.

Q3. Why three algorithms in the ML model?

Random Forest is robust, Gradient Boosting is highly accurate, Logistic Regression is interpretable. The ensemble's weighted average is more accurate than any single one (bias-variance trade-off).

Q4. How does the model "learn"?

During `fit()`, each algorithm adjusts its internal parameters to minimise prediction error on labelled training data. Trees split on the most informative features; logistic regression adjusts weights via gradient descent.

Q5. What features does the model use?

amount, country (encoded), merchant_category, device_type, hour-of-day (cyclic), is_card_present. Country and merchant become one-hot vectors; hour becomes sin/cos pair; amount is standard-scaled.

Q6. How is the data stored?

In a single SQLite file `fraud_detection.db` with 6 tables: roles, users, audit_logs, transaction_records, bank_accounts, cards. SQLAlchemy maps Python classes to these tables.

Q7. Why SQLite and not PostgreSQL?

SQLite needs zero configuration — perfect for development and demos. The code reads `DATABASE_URL` at startup, so switching to PostgreSQL is a 1-line change.

Q8. How does real-time work without WebSockets?

We use **Server-Sent Events (SSE)** — a simple HTTP streaming protocol. The browser keeps an open connection; the server pushes new transaction events on it. Lighter than WebSockets and built into HTML5.

Q9. How do you handle a million transactions a day?

Switch SQLite → PostgreSQL, add an index on `processed_at`, run Gunicorn with multiple workers, and put a Kafka stream in front. The ML model itself can score 10,000+ transactions per second on a single CPU.

Q10. How are passwords stored?

Hashed with **scrypt** via Werkzeug, with per-user salt. The plaintext password is never stored anywhere.

Q11. How does face login work?

face-api.js (a JavaScript port of FaceNet) extracts a 128-dim numeric embedding from a webcam frame. We store this on the user record. At login, we capture a fresh embedding and compute cosine similarity; if ≥ 0.6 , the match passes.

Q12. How do you prevent SQL injection?

All queries go through SQLAlchemy ORM, which uses parameterised statements. We never concatenate user input into SQL strings.

Q13. Why is the rule engine weighted only 35%?

The ML model has seen far more data than any individual rule encodes, so it is the primary signal (65%). Rules are a stable safety net and provide explanations; they are not the sole decider.

Q14. How do you avoid false positives that annoy real customers?

By tuning the thresholds: only scores ≥ 0.85 lead to a decline; scores 0.65-0.85 only trigger a review (analyst can approve in seconds). The recall vs. false-positive trade-off is monitored continuously.

Q15. How do you retrain the model on new data?

Run `python main.py --mode train --data <new_csv>`. The new `fraud_model.pkl` is saved and picked up on next app restart. In production this is scheduled weekly via cron.

Q16. What if the model file is missing or corrupt?

`main.py` checks for it at startup, and if it cannot be loaded, automatically retrains from synthetic data so the system never starts in a broken state.

Q17. How is admin access protected?

RBAC via the `@admin_required` decorator + a global middleware that re-checks role on every request. The `audit_log` records every admin action.

Q18. How do you handle GDPR / data deletion?

Cascade deletes are configured: removing a user removes their accounts, cards. Audit logs are deliberately kept (denormalised `username` column) so the trail survives even after deletion — that's a regulatory requirement, not a bug.

Q19. Where would you deploy this in production?

As a Docker container behind nginx on AWS ECS / GKE / Azure Container Apps, with a managed PostgreSQL (RDS), Redis for sessions, S3 for model artefacts, and CloudWatch / Grafana for monitoring.

Q20. What is the biggest limitation today and how would you fix it?

Current ML uses only the current transaction's features — no user history. Adding an LSTM/Transformer that consumes the user's last N transactions (sequence model) would dramatically improve detection of account takeovers.

19. Conclusion

Sentinel is a complete, production-style fraud-detection platform that brings together **software engineering, data engineering, and machine learning** in a single coherent system. It demonstrates:

- How a modern Python web stack (Flask + SQLAlchemy) supports a multi-tier application.
- How a hybrid ML+rules approach satisfies both accuracy and explainability requirements.
- How role-based access control, audit logging and PCI-DSS-style PAN handling create an enterprise-grade security posture.
- How real-time UX is achieved with simple HTTP-based streaming.
- How clean module boundaries (auth / api / admin / banking / models / real_time) keep a 13,000-line codebase maintainable.

The roadmap (deep learning, graph models, federated learning, mobile app) shows clear paths to scale this from a prototype to a multi-tenant, regulator-grade platform.